

Azure Quantum is the other aggregator, with IonQ, Quantinuum, Rigetti and Atom Computing among its backends, plus Microsoft's own Q# and the QDK. Sensible if you are already an Azure shop.

Then there are the independents worth knowing. qBraid and Strangeworks sit above the hyperscalers and give you one interface across several of them. D-Wave Leap is a different animal entirely and I will come back to it.

There is no Indian public cloud offering QPU access at the time of writing, though there is domestic hardware work underway under the National Quantum Mission. For now, expect your jobs to execute on machines in the US or Europe. If your organisation has data residency rules, raise this early with legal rather than late, because it has stopped more than one pilot I have been involved with.

Annealing is not the same thing

This is worth being blunt about, because the confusion is common and expensive.

D-Wave's machines are quantum annealers. They do not run gate-model circuits. They find low-energy configurations of a problem you have expressed as a QUBO or Ising model,

which makes them a fit for certain optimisation shapes and useless for Shor's algorithm or quantum chemistry.

The practical difference for a developer: you write completely different code, using the Ocean SDK rather than Qiskit, and you cannot port between them. D-Wave has offered free metered access for years and their machines have far more qubits than any gate-model device, which is exactly why the qubit-count comparison misleads people. Different technology, different problem class.

If your problem is genuinely combinatorial optimisation, an annealer is worth an afternoon. Spend some time on running the same problem through a good classical solver like OR-Tools first, so you have something to compare against.

The open source layer

This is where the field is healthiest, and it is genuinely the most encouraging part of the whole picture. Almost every serious tool is open source and permissively licensed.

Qiskit is the centre of gravity. Apache 2.0, Python interface over a Rust core, and since version 2.0 there is a C API as well for people

writing extensions or embedding it. Version 2.3 landed in January this year. The programming model is built around circuits, operators and two primitives: Sampler, which gives you measurement outcomes, and Estimator, which gives you expectation values of observables. Most tutorials you find will use one or the other. It also carries the most capable transpiler in the ecosystem, which matters more than beginners expect.

Cirq is Google's framework, Apache 2.0, built with superconducting device constraints close to the surface. Strong for research and algorithm prototyping.

PennyLane from Xanadu takes a different angle. It treats a quantum circuit as a differentiable function, so you can compute gradients through it and plug the whole thing into PyTorch or TensorFlow. If your interest is quantum machine learning or variational algorithms, start here rather than with Qiskit.

pytket from Quantinuum is a compiler-first toolkit with a reputation for aggressive circuit optimisation. If you are trying to squeeze a circuit under the depth limit of a real device, it is worth benchmarking against Qiskit's transpiler.

Ocean is D-Wave's suite for annealing work. **CUDA-Q** from Nvidia targets GPU-accelerated simulation and hybrid workloads. **Qibo** covers simulation and self-hosted hardware control. **Perceval** is Quandela's photonic framework.

And **OpenQASM** works underneath all of it as the interchange format, which is why cross-framework transpilation is possible at all.

A word on simulators, since this is where you will spend most of your time. Classical simulation of a quantum circuit costs memory that doubles with each qubit added. Around thirty qubits is comfortable on a decent workstation. Beyond forty you need serious hardware. Past roughly fifty, full state-

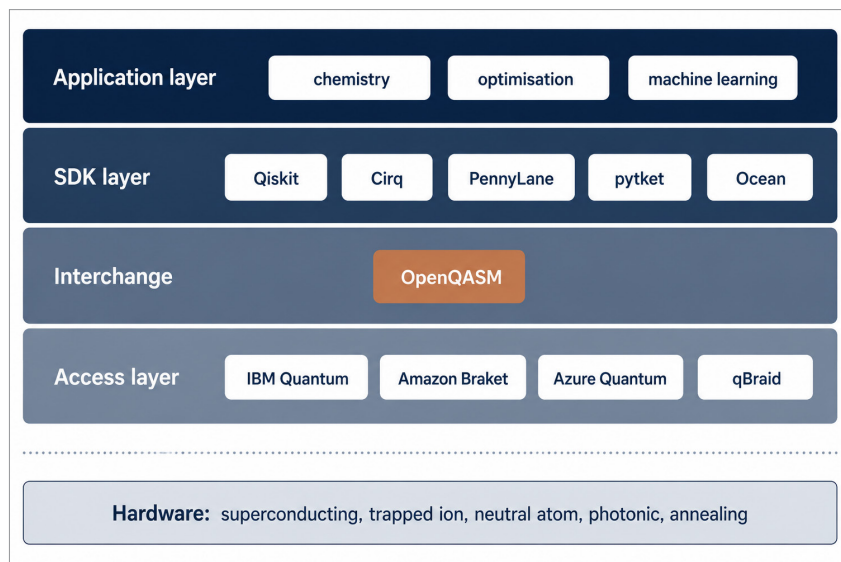


Figure 2: The architecture